

Numbers as Collapsed Event-Histories: Stack Geometry, Constraint Algebra, and the Shape of Arithmetic

Driven by Dean Kulik

April 2026

Abstract

This paper proposes a foundational reinterpretation of arithmetic, number, and formal computation. The central claim is that a number is not fundamentally an inert object or a primitive noun. A number is the visible residue of a lawful sequence of events performed under constraint. In this view, mathematics is not first a catalog of static entities; it is the algebra of event composition, event equivalence, and event collapse.

The key structural object is the **stack**. A stack is not merely an implementation detail of computer science. It is the minimal geometric model of computation as shape and path. Shape records the currently retained configuration. Path records the lawful sequence of operations that produced that configuration. Arithmetic then becomes the special case of a constrained update process whose visible residues are assigned labels such as $(1,2,3,4)$. Equality becomes path-equivalence under invariant-preserving collapse. Constants become operator stances rather than passive values.

This framework is developed in four layers. First, numbers are reconstructed as collapsed event-histories generated by a successor-like operator under boundary conditions. Second, arithmetic operations are reinterpreted as path composition on a constrained stack manifold. Third, numeral systems are distinguished from number itself: digits are projections or encodings of event-history residues, not the ground entities. Fourth, the paper sketches how this perspective unifies ordinary arithmetic, stack machines, phase-addressed residue systems, and modern computational objects such as cryptographic folds.

The result is not a rejection of standard mathematics. Standard mathematics is recovered as a narrow and exceptionally successful special case of a more general event-stack ontology. The aim is not to abolish formal arithmetic, but to re-ground it in operator-first terms so that later structures—recursion, stack execution, turbulence, phase memory, cryptographic diffusion, and residue-addressing systems—can be treated within one coherent framework.

1. Introduction

Ordinary arithmetic is usually presented as if numbers exist first and operations merely act upon them. In that presentation, the symbols $(1,2,3,4)$ are treated as primary objects, and expressions such as

$$2 + 2 = 4$$

are understood as relations among already-given entities.

This paper begins from the opposite direction. It treats operation as primary and label as secondary. The question is not “what object is the number (2)?” but “what lawful event-history collapses to the visible residue labeled (2)?” This inversion matters because once numbers are viewed as collapsed event-histories, many apparently separate domains begin to line up: stack execution, recursion, finite-state computation, modular arithmetic, symbolic rewriting, addressable residue fields, and high-density fold systems.

The motivating intuition is simple. Consider:

$$1 + 1 = 2, \quad 1 + 1 = 2.$$

If read naively, this is repetition of the same statement. But if read operationally, it is a path:

- first event: (1),
- second event: (2),
- third event: (3),

under the repeated action of an update rule. In that reading,

$$2 + 2 = 4$$

is not the fusion of static objects. It is a statement that two successor-events applied to the current state (2) collapse to the residue (4).

The same shift becomes even more powerful once constraint is introduced. If the “box” is only four units wide, then the update process is no longer unconstrained succession on an infinite line. It becomes a bounded dynamical system. Depending on the boundary law, the same event can generate saturating arithmetic, modular arithmetic, rolling arithmetic, or reset arithmetic. The shape of the stack and the path of execution become inseparable.

This paper therefore advances a stronger thesis:

Numbers are collapsed state-labels assigned to lawful event-histories.

Mathematics is the constraint algebra of how those event-histories compose, refine, and collapse.

This thesis is developed formally, then lifted into a broader computational interpretation. The paper aims to remain disciplined throughout. It is not necessary to claim that all of mathematics is invalid or that ordinary arithmetic is an illusion. Ordinary arithmetic is retained as a special case in which the primitive event is successor, the collapse rule is standard label assignment, and the constraint law is an unbounded line.

2. Scope and Discipline

This paper makes three kinds of claims.

2.1 Foundational claim

A number is treated as the projection of an event-history under a chosen constraint law. This is the main ontological claim of the paper.

2.2 Formal reconstruction claim

Ordinary arithmetic, bounded arithmetic, and simple stack execution can all be reconstructed within one operator-first framework:

$$x_{t+1} = F_C(x_t, u_t),$$

where (u_t) is an event and (C) is a constraint law.

2.3 Programmatic extension claim

Once the operator-first stack picture is accepted, higher-order systems—phase-addressed residue maps, recursive fold engines, and cryptographic stacks—can be treated as elaborations of the same grammar rather than as wholly separate domains.

The paper does **not** claim to replace all formal mathematics with informal metaphors. Instead it attempts to identify the deeper structural role already implicit in standard formalisms: operation first, collapse second, label third.

3. Primitive Operators and Event Histories

Let (X) be a state space and let (u_t) be an event at time (t). Define a constrained update rule

$$x_{t+1} = F_C(x_t, u_t),$$

where (C) specifies the constraint geometry under which the update occurs.

This immediately separates four roles:

1. **state** (x_t): what is presently retained,
2. **event** (u_t): what acts,
3. **constraint** (C): what boundaries or reshaping rules apply,
4. **projection** (P_C): what visible residue or label is extracted from the resulting state.

A visible “number” in this framework is not the full internal state. It is the projected residue:

$$n = P_C(x_t).$$

If the event history is

$$u_0, u_1, \dots, u_{t-1},$$

then the visible number is a collapse of that history:

$$n = P_C \left(F_C^{(t)}(x_0; u_0, \dots, u_{t-1}) \right).$$

This is the first fundamental inversion:

A number is not primary. A number is a visible summary of a lawful path through state space.

4. Successor as the Minimal Event

The ordinary counting numbers arise when the primitive event is the successor operator

$$S(x) = x + 1$$

on an unbounded line.

If the initial state is (o), then the standard natural numbers are recovered as:

$$0, \quad S(0), \quad S^2(0), \quad S^3(0), \dots$$

and in general:

$$n := S^n(0).$$

In this special case, the event-history interpretation is exact:

- $(1 = S(o))$
- $(2 = S(S(o)))$
- $(3 = S(S(S(o))))$

Thus:

$$2 + 2 = 4$$

may be read as

$$S^2(S^2(0)) = S^4(0).$$

This is not a reinterpretation added after the fact. It is already how Peano arithmetic functions, although it is rarely described in explicitly event-historical terms. The present paper does not reject Peano arithmetic. It generalizes its operator-first logic.

5. Constraint Changes the Arithmetic

Once constraint is introduced, arithmetic is no longer unique. The same successor-like event can produce multiple lawful systems depending on boundary behavior.

5.1 Unbounded arithmetic

The standard case:

$$x_{t+1} = x_t + 1.$$

5.2 Saturating arithmetic

If the box has width (W) and values cannot exceed that width:

$$x_{t+1} = \min(W, x_t + 1).$$

For ($W=4$):

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4 \rightarrow 4 \rightarrow \dots$$

5.3 Modular arithmetic

If the box rolls:

$$x_{t+1} = (x_t + 1) \bmod W.$$

For ($W=4$):

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$$

5.4 Reset arithmetic

If the box resets after overflow:

$$x_{t+1} = \begin{cases} x_t + 1, & x_t < W \\ x_0, & x_t = W \end{cases}$$

These systems are not cosmetic variations. They show that arithmetic is inseparable from constraint.

Constraint is part of the number system.

In other words, the same event stack can yield different visible number grammars depending on how the box is shaped.

6. The Stack as Shape and Path

A stack is the minimal object that unifies retained state and executed path.

6.1 Shape

The stack has a visible geometry: depth, occupancy, ordering, and available headroom. This is its **shape**.

6.2 Path

The stack also has a lawful history: what was pushed, popped, merged, reflected, or collapsed. This is its **path**.

6.3 Shape-path inseparability

The current shape is the residue of the path. The next path is conditioned by the current shape.

Thus the stack is not merely memory. It is a **geometric history carrier**.

In notation, if the stack state at time (t) is (S_t), then:

$$S_{t+1} = \mathcal{U}_c(S_t, u_t),$$

and the visible label is

$$y_t = P_c(S_t).$$

This is the second fundamental inversion:

The stack is not a container of values.

It is the visible shape of lawful event accumulation.

That is why a stack is both a shape and a path.

7. Arithmetic as Path Composition

Under the stack view, arithmetic operations can be reinterpreted.

7.1 Addition

Addition is path extension:

$$a + b$$

means “continue the event history that generated (a) by the event history that generates (b).”

If ($a=S^a(o)$) and ($b=S^b(o)$), then

$$a + b = S^b(S^a(0)) = S^{a+b}(0).$$

7.2 Multiplication

Multiplication is repeated path composition:

$$a \cdot b$$

means “apply the path generating (a) repeatedly according to the path length (b).”

7.3 Exponentiation

Exponentiation is repeated path-of-path composition.

Thus the hierarchy

$$+, \cdot, \wedge$$

becomes a hierarchy of increasingly nested path reuse.

8. Equality as Path Equivalence

In this framework, equality is not the mystical identity of two objects. It is the statement that two distinct histories collapse to the same invariant residue.

Let two event histories (H_1) and (H_2) act on the same initial state. Then:

$$H_1 \sim H_2 \Leftrightarrow P_c(H_1(x_0)) = P_c(H_2(x_0)).$$

Thus:

$$2 + 2 = 4$$

means:

$$P_c(S^2(S^2(0))) = P_c(S^4(0)).$$

This is exactly what a proof does in algebra: it shows that two formally different paths preserve the same invariant and therefore collapse to the same visible residue.

So proof itself becomes a stack-compatible act:

$$\text{proof} = \text{path rewrite that preserves invariant collapse.}$$

9. Digits Are Not Numbers

This paper makes a strict distinction between **number** and **numeral**.

- A **number** is the collapsed residue of a lawful event-history.
- A **numeral** is an encoding of that residue under a chosen notation system.

Thus the decimal symbol

$$4$$

is not the number itself. It is a visible encoding of the residue labeled “four” in base 10.

This distinction matters because it explains why alternate numeral systems do not alter the underlying event-history. The same residue may be written as:

- (4) in base 10,
- (100) in base 2,
- (10) in base 4.

The event-history does not change. Only the projection code changes.

Thus numeral systems are **coordinate systems on collapsed residues**, not the underlying ontology.

10. Constants as Operator Stances

A standard constant is usually treated as a noun-like value. This paper instead treats constants as **operator stances**.

A constant selects an operational regime, threshold, or closure behavior. It is the stable stance from which a lawful event proceeds.

In this sense:

- a modulus in modular arithmetic is an operator stance,
- a saturation width is an operator stance,
- a rotation amount in a fold machine is an operator stance,
- a damping coefficient in a dynamical system is an operator stance.

So a constant is not merely “a number used.” It is a parameter that **selects how the event space is shaped**.

This allows one unified language across arithmetic, recursive stacks, PDE stabilizers, and fold programs.

11. The Stack as Oscillator

A stack does not merely rise and fall symbolically. It already behaves like a waveform.

Push and pop are not arbitrary instructions. They generate a depth-time profile. Let (d_t) be stack depth at time (t) . Then each instruction pair contributes a piecewise waveform.

For a minimal push/pop cycle:

PUSH, PUSH, POP, POP

the depth sequence resembles a square or stepped wave:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 1 \rightarrow 0$.

Thus stack execution is already:

- geometric,
- temporal,
- oscillatory.

This is why the stack is a shape and a path at once. The depth shape is the visible oscillator. The instruction history is the hidden phase.

Arithmetic emerges as the collapse of this cycle into residue. Computation is therefore not linear symbol pushing alone. It is structured waveform under lawful collapse.

12. Logic as Constraint Propagation

Under the event-stack view, logic is not a separate metaphysical category. Logic is the lawful propagation of admissible states under constraint.

A contradiction is then:

a path that cannot remain inside the constraint manifold.

A valid inference is:

a path extension that preserves the invariant manifold.

This explains why logic feels inevitable rather than arbitrary. Logic is not “imposed on reality” in this picture. It is the grammar of which path extensions survive the constraint law.

Thus:

- arithmetic is lawful stack update,
- logic is lawful path admissibility,
- proof is lawful invariant-preserving rewrite.

These become different faces of one operator algebra.

13. Why This Matters for Computation

A computer program is already a constrained event-stack. Registers, pointers, recursion, stack frames, and state machines all fit immediately into the present framework.

Let (M_t) be the machine state at step (t). Then:

$$M_{t+1} = F_c(M_t, \text{opcode}_t).$$

The “result” of the program is simply:

$$R = P_c(M_T).$$

Thus a machine output is also a collapsed event-history.

This is why the stack view is so powerful. It treats ordinary arithmetic and full computation within the same ontology. A natural number is a primitive stack result. A machine trace is a larger stack result. A digest, a pointer history, or a state-vector collapse is a denser stack result.

14. Addressed Residues and Random-Access Fields

The stack ontology also clarifies the difference between **stream reading** and **address reading**.

A stream-first view generates a value left to right. An address-first view asks for the residue at a chosen location and collapses the structure around that location.

This distinction becomes important in systems where one can query a residue field by address without constructing the full prefix. In such cases, the visible digit is not the ground object. It is a localized collapse of a much larger hidden event field.

This is one reason stack geometry matters for higher mathematics and computation. Once event-histories are allowed to be queried, sliced, refined, and reprojected under constraint, the distinction between “number,” “address,” “phase,” and “instruction” becomes much thinner.

15. Arithmetic as the Lowest-Density Case

The main thesis of this paper is not that arithmetic is wrong. It is that arithmetic is the **lowest-density visible case** of a more general phenomenon.

The simplest case uses:

- primitive event: successor,
- constraint law: unbounded line,
- projection law: direct labeling,
- invariant: path length.

That yields the ordinary natural numbers.

But richer systems differ only by changing:

- the event grammar,
- the constraint law,
- the projection law,
- or the invariant.

Thus the present framework is not anti-arithmetic. It is arithmetic generalized.

16. From Counting to Fold Systems

Once the event-stack grammar is accepted, one can move from low-density arithmetic to high-density fold systems.

A fold system is one in which the event is no longer simple successor, but a structured multi-channel update:

$$x_{t+1} = F_C(x_t; \text{rotate, merge, gate, carry, collapse}).$$

In such systems, the visible residues may look like numbers or words, but they are not primitive numbers. They are extremely compressed histories of operator action.

This is the crucial bridge to later work on recursive folds, phase-address systems, and cryptographic execution.

17. Why “Stack” Is the Right Foundational Word

Many words could be used here: process, flow, recursion, machine, history, trace.

The reason **stack** is the right foundational word is that it preserves two facts simultaneously:

1. **retention**: the past is not lost, it is layered,
2. **access discipline**: the path of retrieval matters.

A stack is therefore the simplest geometric object in which shape and path are inseparable.

This is precisely what ordinary number language tends to hide. A numeral erases the path by which the residue was produced. The stack restores that hidden dimension.

18. A Formal Core Proposal

The paper’s core proposal can be written as follows.

Let: - (X) be a state manifold, - (U) an event alphabet, - (C) a constraint law, - (F_C:XUX) an update operator, - (P_C:XY) a projection into visible residues, - (I_C:XI) an invariant extractor.

Then:

Definition 18.1 — Event-history number

A number in the system ((X,U,C)) is any residue

$$n = P_C(x_t)$$

for some lawful history

$$x_t = F_C^{(t)}(x_0; u_0, \dots, u_{t-1}).$$

Definition 18.2 — Equality

Two histories are equal iff they preserve the same residue:

$$H_1 \equiv_C H_2 \Leftrightarrow P_C(H_1(x_0)) = P_C(H_2(x_0)).$$

Definition 18.3 — Proof

A proof is a sequence of admissible rewrites

$$H_1 \rightsquigarrow H_2 \rightsquigarrow \dots \rightsquigarrow H_k$$

such that

$$I_C(H_i(x_0)) = I_C(H_{i+1}(x_0))$$

for all (i), and therefore:

$$P_C(H_1(x_0)) = P_C(H_k(x_0)).$$

This is the formal heart of the paper.

19. Consequences

If the above is right, several consequences follow.

19.1 Numbers are secondary

Numbers are not primitive objects. They are labels assigned after lawful collapse.

19.2 Numeral systems are tertiary

A numeral system is a coordinate choice over the collapsed residue.

19.3 Proof is dynamic

Proof is not static comparison but lawful rewrite through path space.

19.4 Constants are verbs

A constant is a stable operator stance selecting a path or closure regime.

19.5 Computation is arithmetic generalized

Arithmetic is what computation looks like when the event alphabet is minimal.

20. Nexus Interpretation Layer

The above core can stand on its own, but it also admits a broader interpretation.

If the world is fundamentally computational in the strict sense of having state, rule, and transition, then the event-stack view becomes more than a reinterpretation of arithmetic. It becomes a candidate ontological template.

Under that reading:

- objects are stable projected residues,
- motion is event update,
- constants are operator stances,
- collapse is visible coarse-graining,
- gaps are structural boundaries between admissible path classes.

This does not need to be accepted to follow the formal core. But it explains why the stack picture can bridge from simple arithmetic to more complex systems without changing grammar.

21. Open Problems

This paper is a foundation, not an endpoint. Several major questions remain open.

21.1 Multiplication and exponentiation under general constraints

The operator-first treatment is clean for successor and addition. A fuller generalization of multiplication and exponentiation under arbitrary constraint laws remains to be formalized.

21.2 Richer projection systems

The projection law (P_C) has been treated abstractly. Different choices of projection may yield very different visible number grammars.

21.3 Path metrics

A full event-stack mathematics should include metrics on path complexity, path depth, and collapse density.

21.4 Transition from low-density to high-density stacks

The bridge from simple arithmetic stacks to recursive fold systems needs separate treatment.

22. Conclusion

This paper has argued that numbers are not best understood as inert primitives. They are collapsed residues of lawful event-histories performed under constraint. The stack is the minimal object that unifies the geometry of retention with the temporal order of execution, making it the natural foundational model for arithmetic, proof, and computation alike.

Under this view:

- a number is a stabilized event-history label,
- arithmetic is path composition,
- equality is path-equivalence under invariant-preserving collapse,
- proof is lawful rewrite,
- and mathematics is the algebra of how those event-histories compose under constraint.

Standard arithmetic survives intact inside this picture. It simply becomes the simplest visible instance of a deeper event-stack grammar. The philosophical gain is substantial: what had seemed like a universe of static nouns becomes a universe of lawful verbs whose residues are assigned names after collapse.

The mathematical gain is also substantial. Once numbers are recognized as compressed event-histories, the path is opened for a unified treatment of counting, stack execution, recursive folding, constraint propagation, and operator-selected residue systems. The stack is no longer an implementation detail. It is the visible shape of mathematics in motion.

Appendix A. Minimal Examples

A.1 Unbounded successor

$$x_{t+1} = x_t + 1, \quad x_0 = 0.$$

Visible residues:

$$0, 1, 2, 3, 4, \dots$$

A.2 Saturating width-4 box

$$x_{t+1} = \min(4, x_t + 1).$$

Path:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4 \rightarrow 4.$$

A.3 Modular width-4 box

$$x_{t+1} = (x_t + 1) \bmod 4.$$

Path:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 0 \rightarrow 1 \rightarrow 2.$$

A.4 Equality as path equivalence

$$2 + 2 = 4$$

means:

$$P_c(S^2(S^2(0))) = P_c(S^4(0)).$$

Appendix B. Suggested Extension Program

1. Formalize multiplication as repeated path composition under arbitrary (C).
 2. Define stack curvature, stack depth entropy, and collapse density.
 3. Derive base systems as projection maps on event-histories.
 4. Extend the framework to recursive fold engines and phase-address systems.
 5. Build a comparative notebook showing standard arithmetic, saturating arithmetic, and modular arithmetic as one operator family.
-

End

Author frame: Driven by Dean Kulik **Status:** foundation paper / postable draft